Corte Máximo em Grafos Grandes e Esparsos: Heurística Gulosa com Busca Local

Eduardo Marinho[1], **Marcos Fontes**[1]

[1]Centro de Ciências Exatas e da Terra
Departamento de Informática e Matemática Aplicada (DIMAp)
Universidade Federal do Rio Grande do Norte (UFRN) – Natal, RN – Brasil
DIM0605 – Projeto e Análise de Algoritmos – Prof. Sílvia Maia
Outubro de 2026

Abstract. *This paper presents a greedy heuristic with 1-flip local search for the Maximum Cut problem on large sparse graphs. Vertices are first placed greedily in decreasing order of degree, and a queue-based local search then improves the cut in near-linear time. For non-negative weights, the algorithm guarantees a 1/2 approximation ratio. Experiments on G-set instances and on a 100,000-vertex random graph show that the method reaches 93–96% of the best known value on sparse random graphs and unions of planar graphs and about 80% on toroidal graphs with* ± 1 *weights, in milliseconds.*

Resumo. *Este trabalho apresenta uma heurística gulosa com busca local 1-flip para o problema do Corte Máximo em grafos grandes e esparsos. Os vértices são inicialmente posicionados de forma gulosa, em ordem decrescente de grau, e uma busca local baseada em fila melhora o corte em tempo quase linear. Para pesos não negativos, o algoritmo garante aproximação de razão 1/2. Nos experimentos com instâncias do G-set e com um grafo aleatório de 100 mil vértices, o método alcançou de 93 a 96% do melhor valor conhecido em grafos aleatórios esparsos e em uniões de grafos planares e cerca de 80% em grafos toroidais com pesos* ± 1*, em milissegundos.*

1. Introdução

O problema do Corte Máximo (*Max-Cut*) consiste em dividir os vértices de um grafo em dois grupos de modo que o peso total das arestas entre os grupos seja o maior possível. Ele aparece em áreas como a física estatística e o projeto de circuitos [Burer et al. 2002, Benlic and Hao 2013], mas é NP-difícil. Por isso, a literatura recorre a algoritmos de aproximação, como o de Goemans e Williamson [Goemans and Williamson 1995], que garante razão $\approx 0{,}878$ mas é caro, e a metaheurísticas, que obtêm cortes muito bons sem garantia e podem levar de segundos a horas por instância [Festa et al. 2002, Benlic and Hao 2013].

Em grafos grandes e esparsos, nos quais o tempo de resposta pesa tanto quanto a qualidade do corte, nenhuma das duas opções costuma ser adequada. Este trabalho estuda uma alternativa barata: uma construção gulosa que processa os vértices em ordem decrescente de grau, seguida de uma busca local 1-flip com fila. O algoritmo usa memória $O(n+m)$, roda em tempo próximo do linear na prática e, para pesos não negativos, garante um corte de pelo menos metade do ótimo.

Avaliamos o método em instâncias do G-set [Helmberg and Rendl 2000] e em um grafo aleatório de 100 mil vértices, gerado por nós. Nos grafos aleatórios esparsos e na união de grafos planares, o corte ficou entre 93% e 96% do melhor valor conhecido; nos toroidais com pesos ± 1, em torno de 80%. O grafo maior foi processado em 16 ms.

O artigo está organizado assim: as Seções 2 e 3 definem o problema e discutem sua complexidade, e a Seção 4 revisa as heurísticas existentes. A Seção 5 descreve as instâncias, as Seções 6 e 7 apresentam e analisam o algoritmo, a Seção 8 traz os resultados e a Seção 9 conclui.

2. Descrição do Problema

Seja $G = (V, E)$ um grafo não direcionado com $n = |V|$ vértices e $m = |E|$ arestas, em que cada aresta $uv \in E$ tem um peso inteiro w_{uv}. Denotamos por $N(v)$ o conjunto de vizinhos de v, por $\deg(v) = |N(v)|$ o seu grau, por Δ o grau máximo e por $W = \sum_{uv \in E} |w_{uv}|$ o peso total das arestas. Uma partição dos vértices em dois lados é uma função $s : V \to \{0, 1\}$, e uma aresta é *cortada* quando seus extremos ficam em lados diferentes. O problema pede a partição que maximize o valor do corte,

$$\mathrm{cut}(s) = \sum_{\substack{uv \in E \\ s(u) \neq s(v)}} w_{uv}, \tag{1}$$

e chamamos de OPT o valor ótimo.

Uma forma equivalente, usada nas relaxações da Seção 4, associa a cada vértice uma variável $x_v \in \{-1, +1\}$. Como $(1 - x_u x_v)/2$ vale 1 quando a aresta é cortada e 0 caso contrário,

$$\mathrm{cut}(x) = \sum_{uv \in E} w_{uv} \, \frac{1 - x_u x_v}{2}. \tag{2}$$

Com pesos não negativos, $\mathrm{OPT} \leq W$, e com pesos positivos a igualdade só ocorre em grafos bipartidos. Pesos negativos mudam o problema: cortar uma aresta negativa diminui o valor do corte, e uma partição pode ter valor negativo (como na coluna *Aleat.* da Tabela 1), embora $\mathrm{OPT} \geq 0$, já que colocar todos os vértices no mesmo lado dá corte zero. Neste trabalho o foco são grafos grandes e esparsos, com $m = O(n)$, para os quais interessam algoritmos de custo próximo de linear.

3. Complexidade

A versão de decisão do Max-Cut (existe um corte com valor pelo menos k?) pertence a NP, pois uma partição é verificada em tempo $O(n + m)$, e Karp [Karp 1972] a incluiu em sua lista clássica de problemas NP-completos. Logo, a versão de otimização é NP-difícil. Ao contrário do corte mínimo, que com pesos não negativos é resolvido em tempo polinomial por fluxo máximo, não se espera um algoritmo polinomial exato para o Max-Cut, a menos que P = NP.

Aproximar o problema também é difícil. Håstad [Håstad 2001] mostrou que, a menos que P = NP, nenhum algoritmo polinomial garante razão melhor que $16/17 \approx 0{,}941$. O melhor algoritmo conhecido é o de Goemans e Williamson [Goemans and Williamson 1995], com razão $\approx 0{,}878$, e Khot et al. [Khot et al. 2007]

mostraram que, assumindo a Conjectura dos Jogos Únicos, essa razão não pode ser superada, o que fecharia a lacuna entre os dois valores.

Essas garantias valem para pesos não negativos. Com pesos de sinais mistos, como nos grafos toroidais do G-set, avaliamos a qualidade das soluções de forma empírica, comparando com os melhores valores conhecidos (Seção 8).

4. Heurísticas e Algoritmos Aproximativos

Os métodos para o Max-Cut formam três famílias, de custo e qualidade crescentes. A primeira reúne algoritmos simples com razão $1/2$. A partição aleatória corta cada aresta com probabilidade $1/2$, o que dá um corte esperado de metade do peso total em tempo $O(n)$, e serve de linha de base. A heurística gulosa [Sahni and Gonzalez 1976] insere os vértices um a um no lado que corta mais peso até os vizinhos já colocados, garante pelo menos $W/2$ em tempo $O(n + m)$ e pode ser vista como uma versão determinística da partição aleatória. A busca local 1-flip move um vértice de lado enquanto isso aumenta o corte e termina em um ótimo local que também garante $W/2$ (Seção 7), mas, como só enxerga movimentos de um vértice por vez, pode ficar presa em ótimos ruins.

A segunda família usa relaxação semidefinida. Goemans e Williamson [Goemans and Williamson 1995] trocam cada x_v da formulação (2) por um vetor unitário, resolvem a programação semidefinida (SDP) resultante em tempo polinomial e arredondam a solução com um hiperplano aleatório, obtendo razão esperada $\approx 0{,}878$. O custo da SDP, porém, a torna pouco prática em grafos muito grandes. Burer et al. [Burer et al. 2002] restringem os vetores à dimensão 2, de modo que cada vértice vira um ângulo em uma circunferência. O problema passa a ser uma otimização sem restrições, com uma variável por vértice, bem mais rápida que a SDP completa.

A terceira família são as metaheurísticas, que tentam escapar dos ótimos locais. Festa et al. [Festa et al. 2002] propõem o GRASP, que repete uma construção gulosa randomizada seguida de busca local, e o VNS, que perturba a solução em vizinhanças cada vez maiores. A busca tabu proíbe, por algumas iterações, que um vértice recém-movido volte ao lado anterior, e o *Breakout Local Search* [Benlic and Hao 2013] alterna busca local com perturbações de intensidade adaptativa, sendo nossa referência para os melhores valores conhecidos do G-set. Esses métodos obtêm os melhores resultados, mas levam de segundos a horas por instância.

O nosso algoritmo usa os dois ingredientes mais baratos dessa lista, a construção gulosa e a busca local 1-flip, que também são componentes básicos das metaheurísticas. Aceitamos parar no primeiro ótimo local e, em troca, obtemos tempo próximo do linear e memória $O(n + m)$, com qualidade superior à da construção gulosa sozinha.

5. Banco de Instâncias

O conjunto de instâncias mais usado para testar o Max-Cut é o G-set, criado por Helmberg e Rendl [Helmberg and Rendl 2000], disponível em `https://web.stanford.edu/~yyye/yyye/Gset/`. Ele tem grafos de 800 a 20.000 vértices de vários tipos: aleatórios, uniões de grafos planares e toroidais com pesos $+1$ e -1. Outra coleção conhecida é a Biq Mac Library (`https://biqmac.aau.at/biqmaclib.html`), que não usamos.

Como o tema é grafos esparsos, escolhemos as 12 instâncias do G-set com poucas arestas por vértice (grau médio de 2 a 6) e entre 2.000 e 20.000 vértices: G32, G57, G62, G65, G67, G72, G77 e G81 (toroidais com pesos ± 1), G35 (união de grafos planares) e G55, G60 e G70 (aleatórias). Todas têm valor de referência publicado. Deixamos de fora as instâncias G1 a G22, que são pequenas (800 a 2.000 vértices) e mais densas.

Além disso, escrevemos um gerador próprio (`gerador.cpp`). Ele recebe n, m e uma semente, e sorteia m arestas distintas entre pares de vértices escolhidos de maneira aleatória, todas com peso 1. Com ele geramos a instância R100k, com $n = 10^5$, $m = 2 \cdot 10^5$ e semente 1.

6. Algoritmo Implementado

O algoritmo é dividido em duas etapas. São elas:

6.1. Etapa 1: construção gulosa

Ordenamos os vértices do maior para o menor grau, para decidir primeiro os vértices que influenciam mais arestas. Depois percorremos os vértices nessa ordem. Para cada vértice v, somamos o peso das arestas até os vizinhos que já estão no lado 0 e até os que já estão no lado 1, e colocamos v no lado oposto ao de maior peso. Assim, cada decisão corta o máximo possível de arestas com os vértices já colocados.

6.2. Etapa 2: busca local (1-flip)

O ganho de trocar v de lado é

$$g(v) = \sum_{\substack{u \in N(v) \\ u \text{ do mesmo lado}}} w_{uv} - \sum_{\substack{u \in N(v) \\ u \text{ do outro lado}}} w_{uv}, \tag{3}$$

já que as arestas que não eram cortadas passam a ser, e vice-versa. Enquanto houver um vértice com $g(v) > 0$, trocamos ele de lado. Para não recalcular tudo a cada passo, usamos uma fila: no início todos os vértices estão nela e, quando um vértice muda de lado, só os seus vizinhos voltam para a fila, porque são os únicos cujo ganho mudou. O algoritmo para quando a fila esvazia, ou seja, quando nenhuma troca individual melhora o corte (um ótimo local).

7. Análise do Algoritmo

7.1. Complexidade

A ordenação custa $O(n \log n)$ e a construção gulosa custa $O(n+m)$. Na busca local, cada avaliação de $g(v)$ custa $O(\deg v)$. Como os pesos são inteiros, cada troca aumenta o corte em pelo menos 1, e o corte varia entre $-W$ e W. Então há no máximo $2W$ trocas. Cada troca reenfileira até Δ vizinhos, e reavaliar cada um custa $O(\Delta)$, ou seja, $O(\Delta^2)$ por troca. Somando as n avaliações iniciais, que custam $O(m)$, o pior caso é $O(W \cdot \Delta^2 + n + m)$, na qual Δ é o grau máximo. Em grafos esparsos com Δ pequeno isso é linear em W. Porém, na prática, ocorrem poucas trocas e o tempo fica perto de linear, como demonstrado pelos experimentos. A complexidade de memória é $O(n + m)$.

7.2. Garantia de aproximação

Com pesos não negativos, a solução final é pelo menos metade do ótimo. Na solução final, nenhum vértice tem ganho positivo, ou seja, para todo v, o peso das arestas cortadas que tocam v é pelo menos o peso das não cortadas. Logo, pelo menos metade do peso das arestas de cada vértice está cortada. Somando sobre todos os vértices, cada aresta cortada é contada duas vezes, então $2\,\mathrm{cut} \geq W$, ou seja,

$$\mathrm{cut} \geq \frac{W}{2} \geq \frac{\mathrm{OPT}}{2}, \tag{4}$$

pois o ótimo nunca passa do peso total. Portanto o algoritmo é uma aproximação de razão $1/2$. Quando há pesos negativos essa garantia não vale.

8. Resultados

Os testes foram feitos em um chip M1 (Apple), com o código compilado com `clang++ -O2`. Os tempos medem só o algoritmo, sem considerar o tempo de I/O. A Tabela 1 compara três valores: a partição aleatória, a construção gulosa sozinha e a solução após a busca local. Para o G-set, também comparamos com o melhor valor conhecido na literatura [Benlic and Hao 2013]. Como o R100k foi gerado por nós, não há valor de referência.

Tabela 1. Resultados. Aleat.: partição aleatória; Gul.: só a construção gulosa; G+BL: guloso seguido da busca local; Melhor: melhor valor conhecido.

Inst.	n	m	Aleat.	Gul.	G+BL	Melhor	G+BL/Melhor	Tempo (ms)
G32	2000	4000	40	1026	1154	1410	0,818	0,08
G35	2000	11778	5872	7234	7399	7687	0,963	0,31
G55	5000	12498	6263	9349	9634	10299	0,935	0,70
G57	5000	10000	2	2576	2848	3494	0,815	0,27
G60	7000	17148	8585	12946	13278	14188	0,936	1,16
G62	7000	14000	−92	3478	3920	4870	0,805	0,41
G65	8000	16000	−114	4060	4500	5562	0,809	0,45
G67	10000	20000	−84	4962	5580	6940	0,804	0,53
G70	10000	9999	5003	8816	8949	9595	0,933	1,03
G72	10000	20000	110	5050	5650	6998	0,807	0,49
G77	14000	28000	126	7172	8076	9926	0,814	0,67
G81	20000	40000	296	10178	11382	14060	0,810	0,91
R100k	100000	200000	100100	155526	159645	–	–	16,0

O algoritmo melhora muito a partição aleatória em todas as instâncias, e a busca local sempre acrescenta alguma coisa à solução gulosa, nos nossos experimentos algo entre 2% e 13%, dependendo da instância.

A qualidade da nossa solução depende do tipo de grafo. Nos grafos aleatórios esparsos e na união de grafos planares (G35, G55, G60, G70), chegamos a 93–96% do melhor valor conhecido. Nos grafos toroidais com pesos ± 1, ficamos em torno de 80%. Nesses grafos, a busca local que troca um vértice por vez fica presa em ótimos locais ruins, porque melhorar a solução exigiria trocar vários vértices ao mesmo tempo. As metaheurísticas da literatura chegam aos melhores valores justamente por conseguirem escapar desses ótimos locais, mas gastam de segundos a horas por instância, enquanto o nosso algoritmo leva milissegundos.

9. Conclusão

Implementamos uma heurística gulosa com busca local para o Corte Máximo. Ela é fácil de entender e implementar, roda em tempo quase linear, funciona bem em grafos esparsos enormes e tem garantia de $1/2$ do ótimo para pesos positivos. O ponto fraco são as instâncias com pesos negativos, em que a solução fica cerca de 20% abaixo do melhor conhecido. Uma melhoria natural seria repetir o algoritmo várias vezes com ordens aleatórias ou adicionar perturbações para escapar dos ótimos locais, como faz a Breakout Local Search.

Referências

Benlic, U. and Hao, J.-K. (2013). Breakout local search for the Max-Cut problem. *Engineering Applications of Artificial Intelligence*, 26(3).

Burer, S., Monteiro, R. D. C., and Zhang, Y. (2002). Rank-two relaxation heuristics for MAX-CUT and other binary quadratic programs. *SIAM Journal on Optimization*, 12(2).

Festa, P., Pardalos, P. M., Resende, M. G. C., and Ribeiro, C. C. (2002). Randomized heuristics for the MAX-CUT problem. *Optimization Methods and Software*, 17(6).

Goemans, M. X. and Williamson, D. P. (1995). Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6).

Håstad, J. (2001). Some optimal inapproximability results. *Journal of the ACM*, 48(4).

Helmberg, C. and Rendl, F. (2000). A spectral bundle method for semidefinite programming. *SIAM Journal on Optimization*, 10(3).

Karp, R. M. (1972). Reducibility among combinatorial problems. In *Complexity of Computer Computations*. Plenum Press.

Khot, S., Kindler, G., Mossel, E., and O'Donnell, R. (2007). Optimal inapproximability results for MAX-CUT and other 2-variable CSPs? *SIAM Journal on Computing*, 37(1).

Sahni, S. and Gonzalez, T. (1976). P-complete approximation problems. *Journal of the ACM*, 23(3).